UNIT · 1

INTRO TO C LANGUAGE

* C is a programming language developed at AT&T's Bell Laboratories of USA in 1972. It was designed and written by Dennis Ritchie

* C language is simple, more reliable and easy to use.

* Major Part of popular operating system like Windows, Unix, Linux are still written in C.

* C language is case sensitive language. This means that all language keywords, identifiers, function names, and other variables must be entered with consistent letter capitalization.

* Case sensitivity in C language helps to compile the C program faster.

* Case sensitivity means both upper and lower case characters are treated as different

Structure of C Program

Header	# Include <stdio.h>
main	int main() {
Variable Declaration	int a = 10;
Body	printf ("%d", a);
Return	return 0; }

Ques:- write a program to print 'Hello world !!' ?

Ans:-
```
# include <stdio.h>
void main (void)
{
printf ("Hello world !!" \n");
}
```

Output:- Hello World !!

Character set:- It denotes any alphabet, digit or special symbol used to represent information.

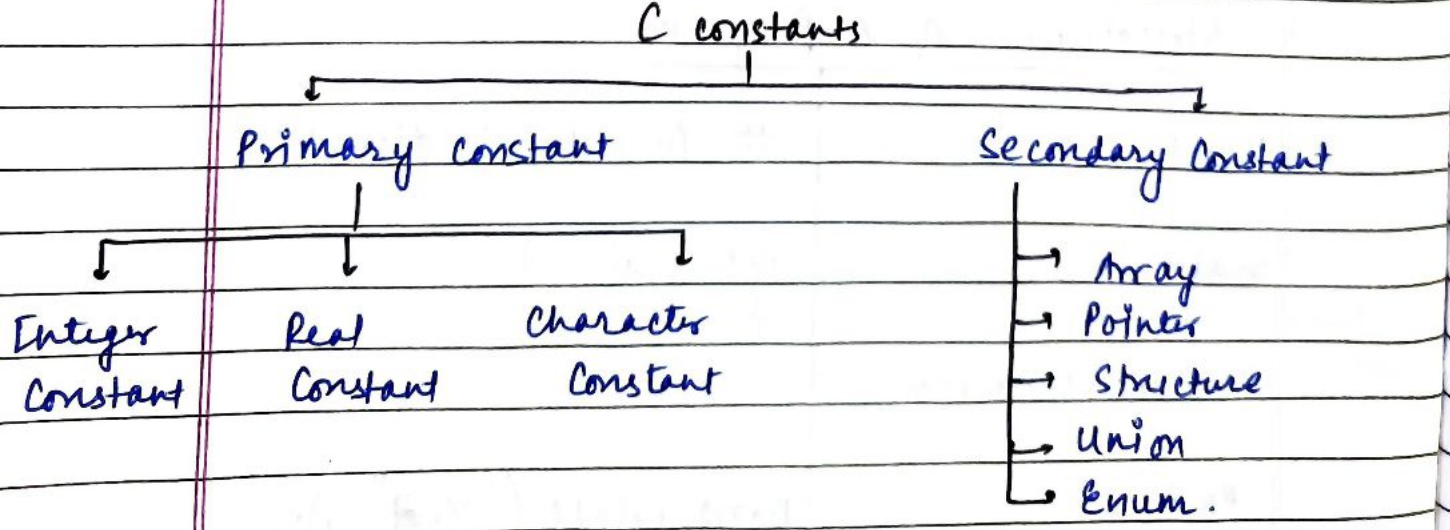
Alphabets: A, B, - - -, Y, Z
a, b, - - -, y, z

Digits: 0, 1, 2, - - -, 9

Special Symbols: ! # % & () - - { } [ ] : ; ' " ? $

Constants, Variables And key words

A constant is an entity that doesn't change during the entire source code.



Rules for Integer Constants

(a) A integer constant must have at least one digit
(b) It must not have a decimal point
(c) It can be either positive or negative
(d) No commas or blanks are allowed.
(f) The allowable range for integer constant is:
-2147483648 to +2147483647 (for VS code & gcc compiler)
-32768 to +32768 (for Turbo C)

Real constants

Real constants are also called as floating point constants. They can be written in two forms - fractional and exponential.

Rules for Real constants.

(a) Must have at least one digit
(b) It must have a decimal point
(c) It could be either positive or negative
(d) Default sign is positive
(e) No comma or blank is allowed
Ex: 325.34 , 426.0 , -32.79

* In exponential form of representation, the real const. is represented in two parts. Parts appearing before 'e' is called mantissa where the part following 'e' is called exponent

Thus 0.000342 can be expressed as 3.4e-4.

Rules:

(a) The mantissa and the exponential part should be seperated by e or E.
(b) Mantissa part may have positive or negative sign.
(c) Default sign of mantissa is positive
(d) The exponent must have at least one digit, which must be positive or negative and default sign is positive
(e) Range of Real constants expressed in exponential form is -3.4e38 to 3.4e38

Ex: +3.2e-5, 4.1e8, -0.2E+3

Character Constants

* It is a single alphabet, single digit or a single special symbol enclosed within single inverted commas. Both the inverted commas should be pointing left.
  Ex: 'A', '5', '='

Variables

* An entity that may vary during program execution is variable.
* Variable names are given to location of memory, these locations can contain integer, real or character constants
* In any language, the type of variable that it can support depend on the type of constant that it can handle.

Ex: An integer variable can hold only integer const. Real variable can hold Real constant & so on.

Variable: It is a name given to a memory location that

Rules for Constructing Variables Names

(a) A variable name is any combination of 1 to 31 alphabets, digits or underscore
(b) The first character in variable name is always a alphabet or underscore
(c) No commas or blanks are allowed within variable name
(d) No special symbol other than underscore (_) can be used in variable name.

Keywords:- Keyword are the words whose meaning has already been explained to the C compiler.

* These keywords cannot be used as variable name.
* Some time these keywords are also known as reserved keywords or reserved words.

There are 32 keywords reserved in C-Language:-

Auto	int
Break	long
Case	register
Char	return
Const	short
Continue	signed
Default	size of
do	static
double	struct
else	switch
enum	typedef
extern	union
Float	unsigned

Data types:-

* It indicates the type of data that a variable can hold. That data may be numeric, alphabatic, symbolic, etc.

* There are broadly two type of data types in C.

(i) Built in data types
(ii) User defined data types.

Built in Data types:- They are basic or primitive data type and designate one single value.
There are four basic fundamental built in data type in C:-

(a) Integer (int):- size in 2 bytes
This is used to indicate an integer no. which is sequence of digits without a decimal point
Ex:- -247, 14020, 27246

(b) Floating point (float):- size in 4 bytes
This is used to indicate the floating point no.
They are same as decimal number but include fractional part as well.
Ex:- $-263.238 = -2.63238\ E+02$ $\{2.63238 \times 10^2\}$
$2.63E-02$ $\{2.63 \times 10^{-2}\} = 0.0263$

(c) Double (double):- size in 8 bytes.
Used to indicate a double precision floating number.
Normally it is equivalent to float, but the number of digits stored after decimal are 16 in double

(e) Character (char) :- size in 1 byte
used to indicate the character type data.
A character in C language is defined as single character enclosed within a pair of apostrophes.
Ex:- 'a', 'p'.

Data type modifier
The basic data type can be modified using a series of type modifiers to fit the requirements of a particular program.

(i) Signed :- This can be applied to integer variables. The defau declaration assumes a signed number. The range of signed integer lies b/w -32768 to 32767

(ii) Unsigned :- It can be used for integer. It can be combined with long or short. If the keyword is unsigned is use before integer the range of positive integers are doubled b negative integers are removed.
Range is from 0 to 65535.

(iii) Long :- This is also used for integers. if the keyword is applied to int, it essentially doubles the length in bits o data type that it modifies.
It can also be used with doubles.
Long int ranges from -2147483648 to 2147483647

(iv) Short :- This makes the size of an int half.
Default declaration of int is short.
Range is same as that of integers in C language

Variable declaration

* If the variable is declared as const variable, its value can't be changed. An attempt to modify its value will result in an error message.
Ex: const float PI = 3.142;

* If the variable is defined as volatile variable or just as variable then its value can be modified at any time. Ex: volatile int x;

* Multiple assignment to the variables is possible in C language. It is possible to assign a value to multiple variables at same time (called as multiple assignment)
Ex: int a=b=c=20;
float x=y=z=3.687;

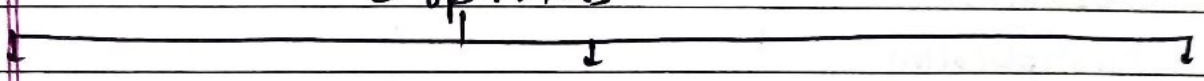
Backlash constants :- It is a comb^n of two characters in which the first character is always backlash (\) and the second character can by any one of a, b, n, t, etc.

* Backlash are used in output statements and are also called escape statement.

Ex:	"\a"	system bell
	"\b"	back space
	"\n"	new line
	"\t"	horizontal tab
	"\v"	vertical tab
	"\0"	null (end of string)

Operators:-

* In order to carry out basic arithmetic operations & logical operations operators are needed.
* Operators act as connectors and they indicate what type of operation is being carried out.
* The values that can be operated by these operator are called operands



Unary operators Binary operators Ternary operators.

Unary operators

An operator that act upon only one operand is known as unary operator. Some unary operators are:-

(i) Unary Minus
(ii) Logical NOT operator
(iii) Bitwise Complementation

Unary Minus:- Any positive operand associated with unary minus operator gets its value changed to negative.

Ex:- a = 3 , b = 4

c = a + (-b) = 3 + (-4) = -1

Binary operators:- These operators act upon operands, hence named as binary. They are further classified into four categories.

(i) Arithmetic Operator
(ii) Logical "
(iii) Relational "

Arithmetic Operators

* They perform basic arithmetic operations such as addition, substraction, multiplication and division.
* Modulus operator is also used in C language that finds the remainder after integer division.

Operation	Operator	Precedence	Associativity
Addition	+	2	Left to Right
Substraction	−	2	"
Multiplication	*	1	"
Division	/	1	"
Modulus	%	1	"

* The precedence indicates which terms to be evaluated.
* Associativity indicates the order in which the terms are evaluated.

Evaluation of Arithmetic Expressions

(i) x * y / z :-

There are two operators (* & /) having same priority. Now expression is scanned from left to right

(a) 1st x * y is evaluated

(b) The result of (x * y) is divided by z.

(ii) a + b * c / d - e.

(a) b * c

(b) (b * c) / d

(c) a + ((b * c) / d)

(d) (a + ((b * c) / d)) - e

Ques: Write a program to illustrate basic arithmetic operators?

Ans:
```
# include <stdio.h>
void main (void)
 {
 int a, b, c;
printf ("Enter a number? ");
 scanf ("%d", &a);
printf ("Enter another number? ");
 scanf ("%d", &b);
 c= a+b;
 printf ("%d\n", c);
 c= a-b;
 printf ("%d\n", c);
 c= a* b;
 printf ("%d\n", c);
 c= a/b;
 printf ("%d\n", c);
 }
```

Ques: Write a program to find the remainder b/w two nos?

Ans:
```
# include <stdio.h>
 void main (void)
  {
  int a=5;
  int b=3;
 int c;
  c= a%b
  printf ("%d\n", c);
  }
```

Output = 2

NOTE: In C language
True output → 1
False output → 0

Relational / Bullion operators :-

* They are used to compare two operands, they also define the relationship existing b/w two constants or variables

* If result is true output is 1
If result is false output is 0.

Operator	Meaning	Precedence	Associativity
<	Less than	1	Left to right
<=	less than or equal to	1	"
>	greater than	1	"
>=	greater than or equal to	1	"
=	assignment operator	1	"
= =	is equal to	2	"
!=	not equal to	2	"

Ques:- Write a code which ask the user to check all bullian operators?

Ans:-
```
# include <stdio.h>
void main (void)
{
 int a=8;
 int b= -5;
printf ("%d < %d is %d \n", a, b, a<b);
printf ("%d == %d is %d \n", a, b, a==b);
printf ("%d != %d is %d \n", a, b, a!=b);
printf ("%d > %d is %d \n", a, b, a>b);
printf ("%d <= %d is %d \n", a, b, a<=b);
printf ("%d >= %d is %d \n", a, b, a>=b);
}
```

Output:-

* 8 < -5 is 0
* 8 == -5 is 0
* 8 != -5 is 1
* 8 > -5 is 1
* 8 <= -5 is 0
* 8 >= -5 is 1

Ques: Write a code which ask the user to enter a no. and check whether it is even or odd!

Ans:-

```
# include <stdio.h>
  void main (void)
  {
  int a, b;
  printf ("Enter a number you want me to check if
                  it is even or odd!");
  scanf (" %d", &a);
  b= a %. 2;
  if (b== =0)
  print f (" Number is even \n");
  else
  print f (" Number is odd \n");
  }
```

Logical Error:- Logical errors are incountered if the solution for a problem does not give the desired results for all test cases.

Syntax Errors:- They are incountered when the rules of programming language are not followed.

Logical operators:-

* They are used to take decisions.
* AND, OR are binary operator & NOT is unary operator.
* Result of these operators is either true(1) or false(0).
* Logical operator are used to connect one or more relational expression.

AND (&&)

Operand 1	Operand 2	Operand 1 && Operand 2
0	0	0
0	1	0
1	0	0
1	1	1

OR (||)

operand 1	operand 2	operand 1 || operand 2
0	0	0
0	1	1
1	0	1
1	1	1

NOT (!)

operand	! operand
0	1
1	0

Ques: Write the code to show the use of
(i) NOT operator
(ii) AND operator
(iii) OR operator.

Ans:- (i)
```
# include <stdio.h>
void main (void)
{
int a=7, b=7, c;
c = !(a!=b);
printf ("%d\n", c);
}
```

Output = 1

(ii)
```
# include <stdio.h>
void main (void)
{
int a, b, c, d;
a= 3;
b= 4;
c= 5;
d= ((a>b) && (b>c));
printf ("%d\n", d);
}
```

output = 0

(iii)
```
# include <stdio.h>
void main (void)
{
int a, b, c, d;
a=7;
b=6;
c=5;
d= ((a>b) || (b>c));
printf ("%d\n", d);
}
```

Output = 1

Ques:- Write a code which ask the user to enter a no. and check if it is divisible by 2 or 3 or not?

Ans:-
```
# include <stdio.h>
void main (void)
{
int n, a, b;
printf ("Enter a number?");
scanf ("%d", &n);
a= n%2;
b= n%3;
if ((a==0) || (b==0))
printf ("Number is divisible by 2 or 3\n");
else
printf ("Number is not divisible by 2 or 3\n");
}
```

Ques:- Ask the user to enter the age of a person to check for the eligibility to vote?

Ans:-
```
# include <stdio.h>
  void main (void)
   {
   int n;
   printf ("Enter a number?");
   scanf ("%d", &n);
   if (n >= 18)
   printf ("person is eligible to vote" \n");
   else
   printf (" person is not eligible to vote \n");
   }.
```

Ques:- Ask the user to enter the percentage of marks secured in class 12th print the result using the following rule.

(i) If Total percentage >= 90 → Distinction
(ii) " " in b/w 70 & 90 → Result is first.
(iii) " " " " 50 & 70 → " " second.
(iv) " " is less than 50 → failed.

Ans:-
```
# include <stdio.h>
 void main (void)
  {
  int n;
 printf (" Enter percentage of class 12 ?");
  scanf (" %. d," & n);
  if (n>90)
 printf (" Distinction \n");
else if((n>=70) && (n <= 90))
 printf (" First \n");
else if ((n>=50) && (n <=70)).
 printf (" Second" \n");
else
  printf (" Fail \n");
 }
```

Increment / Decrement operators :-

* Increment operator is used to increase the value of an integer quantity by one.
This is represented by '++' symbol.
i.e. int a;
a++ or ++a (both are allowed).

* Decrement operator is used to reduce the value of an integer quantity by one
This is represented by '--' symbol.
i.e int b;
b-- or --b (both are allowed).

Difference b/w pre & post increment / decrement.

++a → Pre increment in the value before it is used.
a++ → post increment in the value after it is used.

Ex:-
```
# include <stdio.h>
void main (void)
{
int a=3; b=6;
printf ("a= %.d\n", a++);
printf ("b= %.d\n", ++b);
}
```
Output: a=3
b=7

--b → decrement the value of variable first then use it.
b-- → use the value of variable then decrement it.

Ex: m=6 n=8
--m → 5
n-- → 8

Ques: Write a code to print number from 1 to 10?

Ans:
```
# include <stdio.h>
int main(void)
{
x:                    int i=0;
i++;
printf("%d", i);
if (i<=9)
goto x;                    Output= 1 2 3 4 5 6 7 8 9 10
return 0;
}
```

Ques: Write a code to print number from 10 to 1?

Ans:
```
# include <stdio.h>
int main (void).
{
int i=10;
x:
i--;
printf ("%d\n", i);
if (i>=1)
goto x;
return 0;
}
```

Ques: Write a program to print odd no.s from 1 to 100?

```
# include <stdio.h>
int main()
{
int i=1;
if (i<=10)
{
if (i%2!=0)
}
i++;
goto start;
}
return 0;
}.
```

Ques:- Write a program to print sum of 1st 100 natural numbers?

Ans:-
```
# include <stdio.h>
int main (void)
{
int i=0;
sum;
x:
i++;
sum= sum+i
if (i <=99)
goto x;
printf ("The sum is %d \n", sum);
return 0;
}
```

Ques:- Write a program to check whether a number is prime or composite?

Ans:-
```
# include <stdio.h>
int main (void)
{
int i=0, sum=0, n;
printf ("Enter a number whose factor you want! ");
scanf (" %d", &n);
x:
i++
if (n%.i==0)
sum= sum+i
if (i <=(n-1))
goto x;
if (sum ==2)
else
printf ("%d is composite \n", n);
return 0;
}
```

Ques:- Write a program to print the factors of a number!

Ans:-
```
# include <stdio.h>
int main (void)
{
int n, a;
printf ("Enter a number!\n");
scanf ("%d", &n);
for (a=1 ; a<=n; a++)
{
if (n%a == 0)
printf ("%d is a factor \n", a);
}
return 0;
}
```

Ques:- Write a program to print multiplication table of a no. enter by the user!

Ans:-
```
# include <stdio.h>
int main (void)
{
int num;
printf ("Enter the value of number whose multiplication
        table is to be printed? \n");
scanf ("%d", &num);
for (int i=0 ; i<=10 ; i++)
{
printf ("%d x %d = %d\n", num, i+1, i * num);
}
return 0;
}
```

Ques: Write a program to print odd no.s from 1 to 10?

Ans:
```
# include <stdio.h>
int main (void)
{
int i=1;
x:
if (i<=10)
{
printf ("%d\n", i);
i= i+2;
goto x;
}
return 0;
}
```

Ques: Write a code to print the greatest of two no.s entered by the user?

Ans:
```
# include <stdio.h>
int main(void)
{
int a, b;
printf ("Enter two numbers? ");
scanf ("%d %d", &a, &b);
if (a>b)
printf ("%d is greatest \n", a);
else if (b>a)
printf ("%d is greatest \n", b);
else
printf ("Both are equal \n");
}
```

Ques:- Write a code to print the greatest of three numbers entered by the user?

Ans:-
```
# include <stdio.h>
  int main (void)
  {
  int a, b, c;
printf (" Enter three numbers? ");
scanf (" %d %d %d ", &a, &b, &c);
  if ((a>b) && (a>c))
printf (" %d is greatest \n", a);
  else (b>c)
printf (" %d is greatest \n", b);
  else
printf (" %d is greatest \n", c);
  }
```

Ques:- Write a code to print the greatest of n numbers entered by the user?

Ans:-
```
# include <stdio.h>
  void main (void)
  {
  int n, a, greatest;
printf (" Enter how many numbers you want me to
                                    enter ? ");
  scanf (" %d", &n);
printf (" Enter a number ");
scanf (" %d", &a);
  greatest = a;
n--;
```

```
K:
printf ("Enter a number?");
 scanf ("%d", &a);
if (a> greatest)
 greatest= a;
n--;
 if(n>0)
goto K;
 printf ("The greatest number is %d \n", greatest);
 }.
```

Ques: Write a program to check whether a year is leap or not?

Ans:
```
# include <stdio.h>
  void main (void)
   {
  int a,b,c, d;
  printf ("Enter the year you want me to check?");
   scanf ("%d.", &a);
   b= a%.4;
   c= a%. 100;
   d= a%. 400;
  if (((b==0) && (c!=0)) || ((c==0) && (d==0)))
   printf ("%d is a leap year", a);
   else
   printf ("%d is not a leap year", a);
    }
```

cond^{n}: (i) year %. 4 = 0 and year %. 100 ≠ 0
(ii) year %. 100 = 0 and year %. 400 = 0.

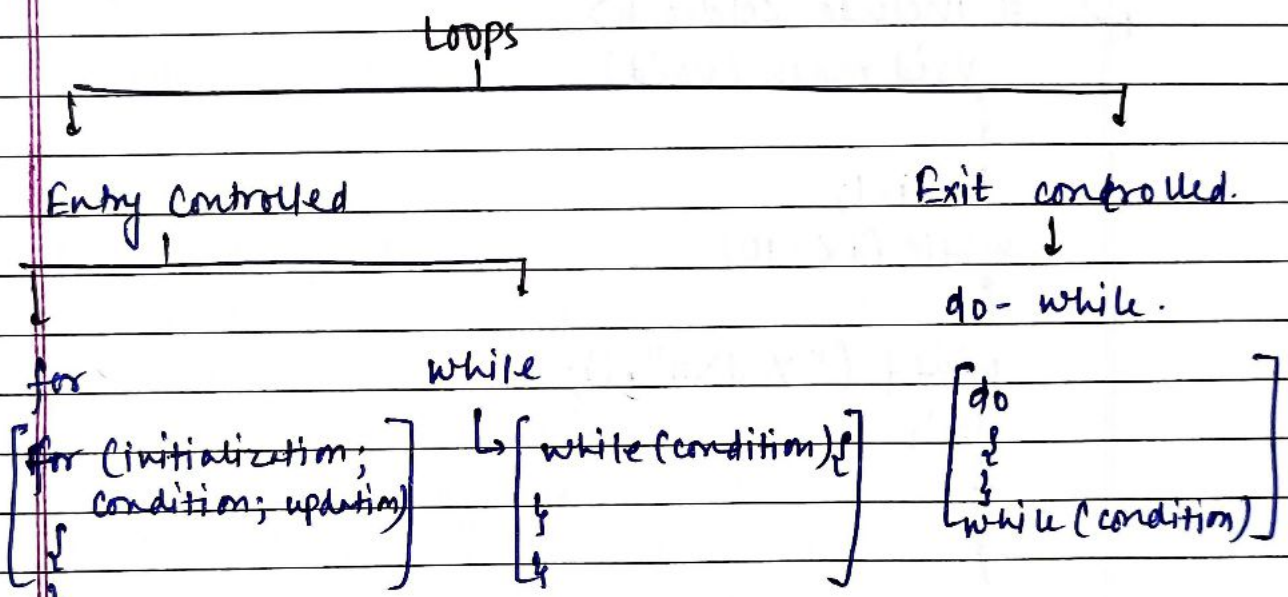
Ques:- Write a program to swap two numbers?
(i) using 3 variables.
(ii) using only two variables.

Ans:- (i)
```
# include <stdio.h>
void main(void)
{
int a, b, c;
printf (" Enter two numbers?");
scanf (" %d %d ", &a, &b);
c = a ;
a = b ;
b = c ;
printf (" The swapped numbers are %d %d \n", a, b);
}
```

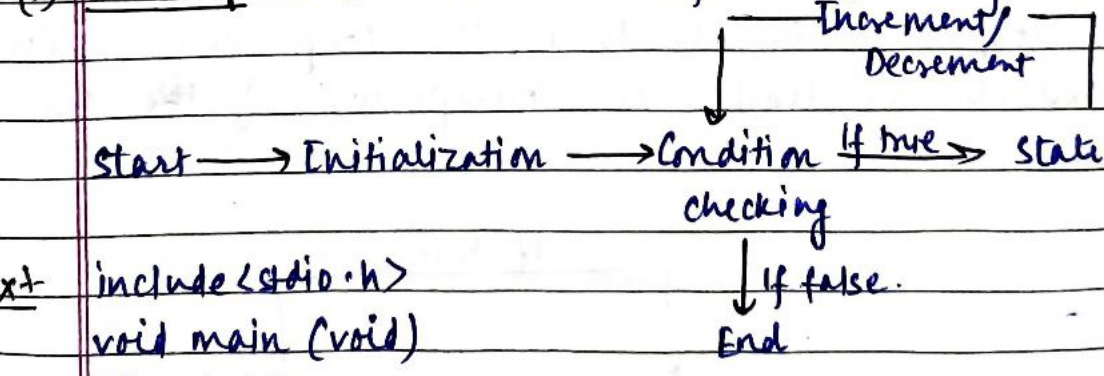
(ii)
```
# include <stdio.h>
void main (void)
{
int a, b;
printf (" Enter two numbers?");
scanf (" %d %d", &a, &b);
a = a+b;
b = a-b ;
a = a-b;
printf (" The swapped numbers are %d %d \n", a, b);
}
```

Looping in C programming

Loops in C programming are used to repeat a block of code until the specified condition is met. It allows programmers to execute a statement or a group of statements multiple times without writing the code again and again.

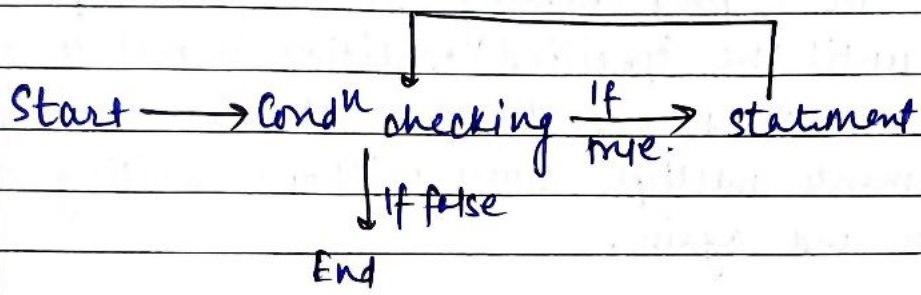


(i) For loop:- Condⁿ is checked before Loop's body executes



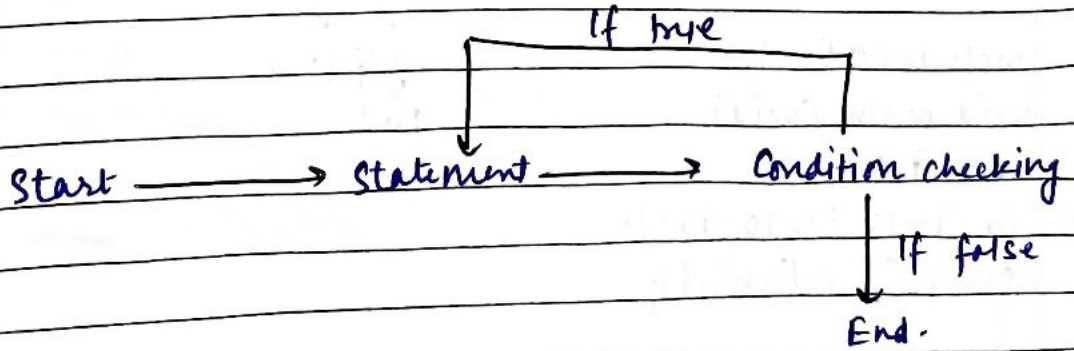
Ex+
```
include<stdio.h>
void main (void)
{  int i;
for (i=1; i<=10; i++).
printf ("%d\n",i);
}
```

(ii) **While Loop**:- Cond^n is checked before entering the body.



Ex:
```
# include <stdio.h>
void main(void)
{
int i=1;
while (i<=10)
{
printf ("%d\n",i);
i+1;
}
}
```

(iii) **Do-while Loop**:- It is an exit controlled loop, which means that the condition is checked after executing the loop body. Due to this, the loop body will execute at least once irrespective of the test condition.



Ex:-
```
# include <stdio.h>
void main (void)
{
int i=1;
do
{
printf(" %d\n", i);
i++;
}
while (i<=10);
}
```

Infinite Loop:- It is executed when the test expression never becomes false, and the body of the loop is executed repeatedly.
A program is stuck in an infinite loop when the condition is always true.
Mostly this is an error that can be resolved by using loop control statements.

Loop control statement:- They are used to change execution from its normal sequence.

(i) break:- It is used to terminate the loop statement.

(ii) continue:- It skips the remaining body and jumps to the next iteration of the loop.

(iii) goto:- It transfers the control to the labeled statement.

1 byte = 8 bit
1 nibble = 4 bit
kilobyte = 1024 bytes = 2^{10} bytes
Mega byte = 2^{20} bytes
Gigabyte = 2^{30} bytes
Terabyte = 2^{40} bytes

Convert Decimal to Binary

Step I:- Divide the given decimal no. by 2 and note down the remainder.

Step II:- Now, divide the obtained quotient by 2 and note down the remainder again.

Step III:- Repeat the above steps until you get 0 as quotient

Step IV:- Now write the remainders in such a way that the last remainder is written first, followed by the rest in the reverse order.

Ex:- Convert 13 into binary form.

$13 \div 2 = 6$ (Remainder = 1)
$6 \div 2 = 3$ (Remainder = 0)
$3 \div 2 = 1$ (Remainder = 1)
$1 \div 2 = 0$ (Remainder = 1)

$(13)_{10} = (1101)_2$

Convert Binary to Decimal

Binary no.:- $d_{n-1}\ d_{n-2} \cdots d_3\ d_2\ d_1\ d_0$

Decimal = $2^0 \times d_0 + 2^1 \times d_1 + 2^2 \times d_2 + \cdots + 2^{n-1} d_{n-1}$

Ques:- Write a program to find the maximum and minimum range of input integers in C programming?

Ans:-
```
# include <stdio.h>
void main (void)
{
int a;
for (a=1; ; a++)
 {
 if (a<0)
  break;
 }
printf ("%d %d \n", a, a-1);
 }
```

Output:- -2147483648
2147483647

Ques:- Write a code to enter 10 numbers and calculate their sum and average?

Ans:- M.I:-
```
# include <stdio.h>
void main (void)
{
int n, a, sum=0;
float avg
for (n=1; n<=10 ; n++)
{
printf (" Enter a number? ");
scanf (" %d ", &a);
sum= sum + a;
 }
avg = sum /10.0;
printf ("The sum is %d and average is %f \n", sum, avg);
 }
```

M.T:-

```
# include <stdio.h>
void main (void)
 {
float n, a, sum=0;
float avg;
for (n=1; n<=10; n++)
 {
printf (" Enter a number?");
scanf (" %f", &a);
 sum= sum+a ;
 }
avg=(float)sum /10;
printf (" The sum is %f and average is %f \n", sum,
                                              avg);
 }.
```

Ques:- Write a code to calculate the area and perimeter of the surface?

Ans:-

```
# include <stdio.h>
   void main (void)
    {
   const float pi= 22.0/7;
float radius;
float peri;
 float area;
printf (" Enter a number?");
 scanf ("%f", &radius);
 peri= 2* pi * radius;
area= pi* radius * radius;
printf (" The perimeter is %f\n", peri);
printf (" The area is %f\n", area);
 }
```

Ques: write a code to find roots of quadratic equation?

Ans:

```
#include <stdio.h>
#include <math.h>
void main (void)
{
float a, b, c, d, r1, r2;
printf(" Enter the coefficients?");
scanf(" %f %f %f", &a, &b, &c);
d = (b*b) - (a*c*4);
if (d<0)
printf(" Roots are imaginary\n");
else if (d==0)
{
printf(" Roots are equal\n");
r1 = -b/(a*2);
printf(" %f \n", r1);
}
else
{
printf(" Roots are real and distinct \n");
r1 = (-b + sqrt(d)) / (a*2);
r2 = (-b - sqrt(d)) / (a*2);
printf(" %f %f \n", r1, r2);
}
}
```

Run in Ubuntu by
gcc quadratic.c -lm

sqrt(d) → used for square root
pow(d, .5) → power (d)^0.5 → (d)^1/2

Bitwise Operators

* All the data stored in a computer's memory as a sequence of bits (i.e. 0, 1); some applications needs manipulation of these bits

* To perform bitwise operations, C provides six operators.

* These operators work on char & int variables but not on floating type data.

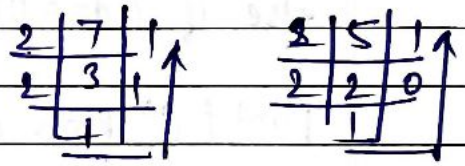
(i) Ampersand (&) → Bitwise AND

7 & 5

111

(&) 101

101 → $(5)_{10}$



(ii) Pipeline (|) → Bitwise OR

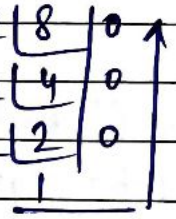
8 | 5

1000

(|) 0101

1101 → $(1)\times2^3 + (1)\times2^2 + 0\times2^1 + 1\times2^0$

$\Rightarrow 8+4+0+1 = (13)_{10}$



(iii) XOR operator (^) :-

$1^1 = 0$

$0^0 = 0$

$1^0 = 1$

$0^1 = 1$

7 ^ 5 → 111

(^) 101

010 → $2^2\times0 + 2^1\times1 + 2^0\times0$

$= 2$

Bitwise Binary operators

(i) Complement operator (~) → Tilde :- If 0 → then 1
If a = $(10)_{10}$ → 1010 If 1 → then 0.

2	10	0
2	5	1
2	2	0
	1	

b = ~(a) → ~(1010) = 0101

Negative no. in Binary :- (i) 1's compliment
(ii) 2's compliment

* ~p = -(p+1) → ex:- ~(-6) = -(-6+1) = 5

(ii) Left shift operator (<<) → shift left by 1 bit
a = 7 → 111
Shift acc → 1110 → 14

* when we left shift by one bit it implies multiplication by 2.

(iii) Right shift operator (>>) :- shift right by 1 bit

a = 7	111	a = 4	100
Shift a>>	011	a>>	010 → 2

* when we right shift by 1 bit it implies dividing a number by 2.

Ques: Write a program to show use of all bitwise operators?

Ans:
```
# include <stdio.h>
void main (void)
{
int a=7;
int b=5;
int c=a+b; /c=a|b; /c=a^b; /c=~a; /c=a<<1; /c=a>>1;
printf ("%d\n", c);
}
```

Ques: Write a program to check if a number is even or odd without using mod(%)?

Ans:
```
# include <stdio.h>
void main (void)
{
int a, temp;
printf ("Enter a number?");
scanf ("%d", &a);
temp = a;
while (temp>=0)
{
temp = temp-2;
}
if (temp==-1)
printf ("%d is odd\n", a);
else
printf ("%d is even\n", a);
}
```

Ques: Write a program to ask the user to enter a character and check if it is a vowel or not?

Ans:-
```
#include <stdio.h>
void main (void)
{
char ch;
printf ("Enter a character ? ");
scanf ("%c", &ch);
if ((ch=='a') || (ch=='e') || (ch=='i') || (ch=='o') || (ch=='u') ||
    (ch=='A') || (ch=='E') || (ch=='O') || (ch=='I') || (ch=='U'))
printf ("Vowel \n");
else
printf (" Consonent \n");
}
```

Ques: Why C is a structured programming language?

Ans:- C is a structured programming language because it supports dividing large problem into smaller, manageable functions (modules), which then used to solve problem.

This approach makes the program easier to understand, maintain, debug and reuse by organizing code into logical, independent blocks that perform specific tasks.

C also uses control structures like loop (for, while) and conditionals (if, switch) to manage the program's flow further contributing to its structured nature.

Reasons :-

(i) Modularity with Functions
(ii) Code Reusability
(iii) Improved Readability and Maintainability
(iv) Structured Control Flow. (v) Top-Down Design

Types of Programming Languages:-

(i) **Machine Language:-** This is the lowest-level language consisting of binary code (0 and 1) directly understood by the computer's CPU. It is extremely difficult for humans to read and write.
It is the fastest programming language.

(ii) **Assembly Language:-** Hardware / platform dependent language.
A low-level language that uses mnemonics (short code) to represent machine instructions.
It is more readable than machine language but still requires a deep understanding of computer architecture.

(iii) **High level language:-** Platform independent language.
These language are designed to be more human-readable and abstract away many of the complexities of computer hardware.
Ex) C language, C++, Python, Java, JavaScript

```
* #include <stdio.h>
  void main(main)
  {
// char ch[20]="AMIT KAPOOR"; // String is an array of characters
// char="AMIT"; This is illegal.
  char ch[20];
  printf("Enter your name?");
  scanf("%[^\n]s", ch); --> no '&' is used in %s
  printf("Hello. %s\n", ch);
  }
```

Ques: Write a code to print the string (Each word on diffnt line)?

Ans:
```
#include <stdio.h>
void main (void)
{
char st[100];
int a;
printf ("Enter a string");
scanf ("%[^\n]s", st);
for (a=0; st[a]!='\x0'; a++)
printf ("%c\n", st[a]);
}
```

* '\x0' → Null character (Enter by the system at the end of the string)

Ques: Differentiate b/w data type casting?

Ans: Implicit Type Casting

* This occurs automatically by the compiler or interpreter without explicit instructions. (Automatic Conversion)

* It typically happens when converting a smaller data type to a larger data type, where no loss of data is expected (safety)
Ex: Assigning an int to a long or an int to a float

* The system infers the need for conversion based on the context of the operation such as an assignment or an arithmetic expressing involving different data type (context driven)

Explicit Data type Casting

* Manual Conversion: This requires the programmer to explicitly instruct a system to convert a value from one data type to another using a casting operator.

* Potential Data Loss:- It is used when converting a large data type to a smaller data type.
Ex:- Casting a double to an int will truncate the decimal part.

* Programmer Control:- The programmer takes direct control over the conversion process, ensuring the value is treated as a specific data type even if it might result in data modification.

* Syntax:- Typically involves a casting operator before the value being converted, or a dedicated conversion function.

Ques:- Write a program to check if a number is even or odd without using any arithmetic operators?
Ans:- Bitwise AND operational logic to be implemented in our program i.e "if A number & 1 == 0" then the number is ever otherwise odd.

eg:- 111 : 7
& 001 : 1
001 ≠ 0 (No. is odd)

eg:- 010 : 2
& 001 : 1
000 = 0 (No. is even)

```
#include <stdio.h>
#include <conio.h>
  void main (void)
  {
int a;
printf ("Enter a number? ");
  scanf ("%d", &a);
```

```
if ((a+1)==0)
printf ("Number is even");
else
printf (" Number is odd");
return 0;
getch();
  }
```

Ques: WAP to reverse the string and check if it is palindrome or not?

Ans:-

```
# include (stdio.h>
# include <string.h>
void main(void)
{
char st[100], nst[100];
int a, b= 0;
printf ("Enter a string?");
scanf ("%.[^\n]s", st);
for (a=0; st[a]!='\x0'; a++);
a--;
for (; a>=0 ; a--)
 {
nst [b]= st[a];
  b++;
}
nst [b]='\x0';
printf (" The reversed string is %s\n", nst);
if (strcmp (st, nst)==0) ——→ To compare two string.
 printf ("Palindrome\n");
else
 printf ("Not a Palindrome\n");
  }
```

NOTE:

ASCII :- American Standard Code for Information Interchange.
↳ 7 bytes (128 characters can be represented)

A → 65
a → 97
0 → 48
space bar
} ASCII codes.

EBCDIC: Extended binary coded decimal Interchange code
↳ 8 bytes (256 characters)

* code to check ASCII values.

```
# include <stdio.h>
void main (void)
{
char ch='0';
printf ("%d %c\n", ch, ch);
}
```

Unicode (universal code):-

```
# include <stdio.h>
void main (void)
{
char ch='a';
for (; ch<='z' ; ch++)
printf ("%d %c\n", ch, ch);
ch='A'
for (; ch<='Z'; ch++)
printf ("%d %c\n", ch, ch);
ch='0'
for (; ch<='9'; ch++);
printf ("%d %c\n", ch, ch);
}
```

*

```
include <stdio.h>
include <string.h>
void main (void)
{
char st1[10]="Hello";
char st2[10]="Hi";
int c;
c= strcmp(st2, st1);
printf("%d\n", c);
}
```

Hello
Hi
same

e f g h i
4

strcmp → use to compare two string

//output = 4

Que: write a program in C to generate the following patterns?

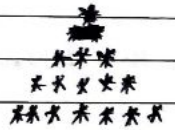
(i)
```
*
**
***
****
*****
```

```
# include <stdio.h>
void main (void)
{
int n, i, o;
printf ("Enter number of rows?");
scanf ("%d", &n);
for (o=1 ; o<=n ; o++)
{
for (i=1 ; i<=o ; i++)
{
printf ("*");
}
printf ("\n");
}
}
```

(ii)

```
#include <stdio.h>
void main (void)
{
int n,i,o;
printf("Enter the number of rows?");
scanf("%d",&n);
for(o=n ; o>0 ; o--)
 {
for(i=1; i<=o ; i++)
 printf(" *");
 printf("\n");
 }
}
```

(iii)



```
#include<stdio.h>
 void main(void)
  {
   int n, i, o, m, blanks;
printf("Enter the number of rows?");
 scanf("%d",&n);
  blanks= n-1;
for(o=1 ; o<=n ; o++)
  {
  for(m=1 ; m<=blanks ;m++)
printf(" ");
 for(i=1; i<=(2*o)-1 ; i++)
   printf("*");
  blanks--;
   printf("\n");
   }
  }
```

(iv)

```
#include<stdio.h>
 void main(void)
 {
int n,i, o, m, blanks;
printf("Enter the number of rows?");
 scanf("%d",&n);
```

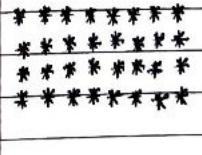
```
blanks= n-1;
 for(o=n; o>0 ;o--)
{
for(m=1 ; m<=blanks ; m++)
 printf(" ");
for(i=1 ; i<=(2*o)-1 ;i++)
 printf("*");
blanks++ ;
 printf("\n");
  }
  }
```

(v)

```
P
PR
PRA
PRAG
PRAGA
PRAGAT
PRAGATI
```

```
#include<stdio.h>
 void main(void)
  {
  int n,i,o;
   char st[30];
printf("Enter your name?");
 scanf("%s",st);
for(n=0; st[n]!='\0' ;n++);
  for(o=0 ; o<n ; o++)
 {
  for(i=0 ; i<=o; i++)
   {
   printf("%c",st[i]);
    }
    printf("\n");
    }
    }
```

(vi)



```
#include<stdio.h>
 void main(void)
  {
  int n,m,i,o;
printf("Enter the number of rows?");
 scanf("%d",&n);
```

```
printf("Enter the number of columns?")
scanf("%d", &m);
for(i=0; i<n; i++)
{
for(o=0; o<m; o++)
printf("*");
printf("\n");
}
}
```

(vii)

```
*****
****
***
**
*
```

```
#include<stdio.h>
void main(void)
{
int n,i,o,m;
printf("Enter the number of rows?");
scanf("%d",&n);
for(i=n; i>=1; i--)
{
for(o=0; o<n-i; o++)
printf(" ");
{
for(m=0; m<i; m++)
printf("*")
}
printf("\n");
}
}
```

(viii)

```
    1
  2 1 2
3 2 1 2 3
4 3 2 1 2 3 4
```

```
#include<stdio.h>
int main()
{
int i,j, n=4
for(i=1; i<=n; i++)
{
for(j=1; j<n; j++)
{
printf(" ");
}
for(j=i; j>=1; j--)
{
printf("%d", j);
}
for(j=2; j<=i; j++)
{
printf("%d", j);
}
printf("\n");
}
return 0;
}
```

(ix)

```
******
*    *
*    *
******
```

```
#include<stdio.h>
void main(void)
{
int i,j,n,m;
printf("Number of rows?");
scanf("%d",&i);
printf("Number of columns?");
scanf("%d",&j); for(n=0; n<i; n++)
for(m=0; m<j; m++)
{
if(n==0||n==i-1||m==0||m==j-1)
printf("*")
else
printf(" ");
}
printf("\n");
}
}
```

(*) Enter your name? AKASH

Output: HSAKA
SAKA
AKA
KA
A

```
# include <stdio.h>
# include <string.h>
int main()
 {
 char name[50];
 int i, j, len;
printf("Enter your name");
 scanf("%s", name)
 len= strlen(name)
printf("%c", name[len-1]);
for(i=len-2 ; i>=0 ; i--)
 {
printf("%c", name[i]);
 }
printf("\n");
for(i=1 ; i<len; i++)
 {
 for(j=0 ; j<i; j++)
 {
 printf(" ");
 }
 for(j=i ; j<len; j++)
 {
 printf("%c", name[len-j-1]);
 }
 printf("\n");
 }
 return 0;
 }
```

* Only int and char can be used in switch(). Constant expression.

Switch case:-

```
# include <stdio.h>
 void main(void)
 {
 int c;
printf("Enter your choice");
 scanf("%d", &c);
 switch(c)
 {
 case 1:
 printf("You entered 1\n");
 break;
printf("You entered 2\n");
 break;
 default:
printf("You entered something other than 1 or 2\n");
 }
 }
```

Program

Ques: Write a which prints numbers from 1 to 100 except those which are divisible by 3, 5 or 7?

Ans:

```
# include <stdio.h>
 void main(void)
 {
 int a;
for(a=1; a<=100 ; a++)
 {
 if((a%3==0)||(a%5==0)||(a%7==0))
 continue;
printf("%d\t", a);
 }
 }
```

{\t → used for horizontal tab.}

Ques:- Write a menu driven program to stimulate a calculator which performs four basic arithmetic operations on 2 floating point numbers entered by the user.

Ans:-
```
# include <stdio.h>
# include <stdlib.h> ——> used when exit is used.
void main (void)
{
float a, b, result;
char choice, dummy;
printf (" Enter two numbers? ");
scanf ("%f %f", &a, &b);
printf (" Enter your choice (+Add, -Minus, * Multiply, /Divide)");
scanf ("%c" &dummy);
scanf ("%c", &choice);
switch(choice)
{
case '+';
result = a+b;
break;
case '-';
result= a-b;
break;
case '*';
result= a* b;
break;
case '/';
result= a/b;
break;
default:
printf ("Invalid choice.\n");
exit (0);
}
printf ("Result is %f\n", result)
}
```

NOTE:-

Array:- It is defined as a collection of elements of the same data type

int a[10] → Array of 10 integers
a[0] → 1st integer a[9] → last integer.

* int a[10] a[10] → 2 dimension order 10×10
a[0][0] → 1st element
a[0][1] → 2nd element
a[9][0] → last row 1st element
a[9][9] → last row 2nd element.

Ex:
```
a[3][3]
1 2 3
4 5 6
7 8 9
```
a[0][0]=1 a[1][0]=4 a[2][0]=7
a[0][1]=2 a[1][1]=5 a[2][1]=8
a[0][2]=3 a[1][2]=6 a[2][2]=9

Ques:- Ask the user to enter two dimensional matrices of order m×n. and then add the two matrices.

Ans:-
```
# include <stdio.h>
void main (void)
{
int row, col, i, j, a[10][10], b[10][10];
printf (" Enter dimensions of matrices");
scanf ("%d %d", &row, &col);
for (i=0; i<row; i++)
{
for (j=0; j<col; j++)
{
printf (" Enter element for A[%d][%d]?", i, j);
scanf ("%d", &a[i][j]);
printf (" Enter element for B[%d][%d]?", i, j);
scanf ("%d", &b[i][j]);
```

```
c[i][j]= a[i][j]+ b[i][j];
 }
}
printf ("Addition of two matrices is:\n");
for(i=0 ; i<row ; i++)
 {
 for (j=0 ; j<col ; j++)
  {
  printf ("%d\t", c[i][j]);
   }
  printf ("\n");
   }
   }
```

Ques: Ask the user to enter a matrix of order mxn and find the transpose?

Ans:

```
# include <stdio.h>
  void main(void)
   {
  int a[10][10] , t[10][10] , row, col, i, j;
printf ("Enter dimension of the matrices?");
  scanf ("%d %d", &row, &col);
  for (i=0; i<row; i++)
    {
    for (j=0 ; j<col ; j++)
     {
printf ("Enter element for a[%d][%d]?", i, j);
  scanf ("%d", &a[i][j]);
  t[j][i]= a[i][j];
    }
    }
  printf ("Transpose of a matrix is:\n");
   for(i=0; i<col ; i++)
    {

  for (j=0 ; j<row ; j++)
    {
```

```
printf ("%d\t", t[i][j]);
 }
printf ("\n");
 }
}
```

USB.C.

Ques: Ask the user to enter two matrice of order mxn & nxo. Perform matrix multiplication?

Ans:

Pointers

* Pointer is a variable that stores the address of other variables but of the same data type.

```
* #include <stdio.h>
  void main (void)
   {
   int a= 32;
   int *p; // Integer Pointer -> addresses of other variables
    p=&a; // p stores the address of a.
   printf ("%d\n", a);
   printf ("%d\n", *p);
   }
```

* Only four arithmetic operations are possible with pointers (+, −, ++, −−)

```
* #include <stdio.h>
   void main(void)
   {
   int a[5]= {1,2,3,4,5};
   int *p;
   p= &a[0];
   printf("%d\n", *p);
   p++; // forwarding a pointer.
   printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
    p++;
    printf("%d\n", *p);
```

```
   p--; // rewinding a pointer.
   printf ("%d\n", *p);
   // *p = p12;
   // printf("%d\n", *p); Illegal */
   for (; *p != 1 ; p--)

   printf ("%d\n", *p);
    }
```

```
* include <stdio.h>
  void main (void)
   {
   int a=32;
   int *p;
   p=0; / p=NULL; (Null points when pointer does not
   printf("%d\n", *p);                point to any value)
   }                  p=&a;
```

We first assign null to the pointer and then assign value to pointer.

Ques: Write an algorithm to print roots of quad. eqn?
Ques: Write an algorithm to print table of a number?
Ques: Write an algorithm to print greatest of three no.?

```
int *p
p=0;
printf("%d\n", *p);
```

Scope of variables

```
#include <stdio.h>
int a=9; // global variable
void main(void)
{
printf("%d\n", a); // prints 9
int a=2; // local variables
a=3;
printf("%d\n", a); // prints 3
if(a==3)
{
int a=7;
printf("%d\n", a); // prints 7
a++; // a becomes 8
}
printf("%d\n", a); // print 3
}
```

* User defined fx^ns

To add two numbers.

```
#include <stdio.h>
void main(void)
{
int a, b, c;
int sum(int x, int y); // function declaration.
scanf("%d %d", &a, &b);
// c= a+b;
//printf("%d %d\n", x, y);
c= sum(a,b); // function call.
printf("%d\n", c);
}
int sum(int x, int y) // function define.
{
return x+y;
}
```

```
* #include <stdio.h>
void main(void)
{
int chkevenodd(int x); // function declaration.
int a, b;
printf("Input a number you want to check?");
scanf("%d", &a);
b= chkevenodd(a); // Call a fx^n by PASSING VALUES
if(b==0)
printf("Even\n");
else
printf("odd\n");
}
int chkevenodd(int x) // function definition.
{
if(x%2==0)
return 0;
else
return 1;
}
```

Storage class specifiers

(i) auto
(ii) static
(iii) extern
(iv) register : fastest to the computer bcoz closest to CPU.

* By default every variable is of the storage class specifier type auto.

eg:
```
#include <stdio.h>
void main(void)
{
auto int a=4;
printf("%d", a);
}
```
output = 4.

```
* #include <stdio.h>
  void main (void)
  {
  int a=7;
  void print (int x);
   print(a);
   print(a);
   print(a);
   }
  void print (int x)
   {
   static int y=1;
   y= y*x
   printf("%d\n", y);
   }
```

Output: 7
49
343

```
* #include <stdio.h>
   void main (void)
    {
   extern int a;
   printf("%d\n", a);
    }
    int a=2;
```

Output = 2.

```
* #include <stdio.h>
   void main (void)
    {
   extern int a;
   void func (void);
   printf("%d\n", a);
    func();
   }
   int a=2;
   void func (void)
   {
   printf("%d\n", a);
   }.
```

To find factorial of a number

```
#include <stdio.h>
  void main (void)
  {
  int a;
  int fact = 1;
  scanf("%d", &a);
  while (a>0);
   {
   fact = fact * a;
    a--;
   }
   printf("%d\n", fact);
   }
```

Ques: write a code in C to check if a number is neon number or not?

Neon Number: If the sum of digits of the square number is equal to the number itself.

eg: $9^2 \rightarrow 81 \longrightarrow 8+1=9$ (9 is a neon number)

only 0, 1 & 9 are neon nos.

Ques: write a code to generate Floyd's Triangle?

```
1
2  3
4  5  6
7  8  9  10
11 12 13 14 15
```

Ans:

```
#include <stdio.h>
  void main(void)
   {
   int n, sum=0, sqn, r;
printf("Enter a number you want me to check if it is
                                  a Neon number!");
scanf("%d", &n);
  sqn= n*n;
```

```
while (sqn!=0)
{
r= sqn%10;
sum= sum+r;
sqn= sqn/10;
}
if (sum==n)
printf ("The no. is a Neon No.\n");
else
printf ("The no. is not a Neon no.\n");
}
```

Ans:-

```
# include <stdio.h>
void main (void)
{
int n, i, j, nat=1;
printf ("Enter no. of rows?");
scanf ("%d", &n);
for (i=1; i<=n; i++)
{
for (j=1; j<=i; j++)
{
printf ("%d ", nat);
nat ++;
}
printf ("\n");
}
}
```

Ques:- (i) WAP to print LCM of two numbers?
(ii) WAP to print LCM of n numbers entered by the user?

Ans:- (i)

```
# include <stdio.h>
void main (void)
{
int num1, num2, lcm, flag=0, r1, r2;
printf ("Enter two no.s whose LCM you want?");
scanf ("%d %d", &num1, &num2);
lcm= num1;
while (flag==0)
{
r1= lcm%num1;
r2= lcm%num2;
if ((r1==0)&&(r2==0))
flag=1;
else
lcm++;
}
printf (" LCM is %d\n", lcm);
}
```

Ques:- Differentiate b/w while and do while with suitable examples.

Ans:- Syntax, Rule of writing.

* while (<condition>)
{
body of the loop
}

* while is dependent on the condition.
* Entry controlled loop.

Ex:-

* do
{
body of the loop
} while (<condition>);
* Do while is guaranteed to executed once.
* Do while is exit controlled loop

Ex:-

Kshitiz s@srmist.edu.in.

Ques: ~~Explain with examples the syntax of for loop?~~

Ans:
```
for (<initialization>; <condition>; <operation>)
 {
 -
 -
 -- body of the loop
 }
```

Syntax of switch case
```
Switch (<var_ name>)
{
case value-1:
-
-
-statements
 break;
case value-2:
-
-
-statements
 break;
 .
 .
 .
default:
-
-
-statements
}
```

Pseudo Code:- Code in our language does not follow rules of a language.

Ex: Start
} Codes in English language.
End.

* Difference b/w Algorithm and Pseudo code?
* Sorting

Ques: Write a program to ~~find~~ enter the no. of notes to give to user for his entered amount?

Ques: write a code to perform linear search on an array?

Ans:-
```
# include <stdio.h>
 void main (void)
 {
 int a[100], search n, i, flag=0;
printf ("How many elements you have in array?");
 scanf ("%d", &n);
 for (i=0; i<n; i++)
 {
 printf ("Enter a[%d]?", i);
 scanf ("%d", &a[i]);
 }
 printf ("What do you want to search in array?");
 scanf ("%d", &search);
 for (i=0; i<n; i++)
 {
 if (a[i]==search)
 {
 printf ("Element found at location %d\n", i);
 flag=1;
 }
 }
 if (flag==0)
 printf ("Element not found\n");
 }
```

Que:- Write a C program to stimulate a cashier giving money to a customer. In the program let the customer enter the amount of value of money. The program prints how many notes of different domination (500, 200, 100, 50, 20, 10, 5, 2, 1) are given to the customer?

Ans:-
```
#include <stdio.h>
void main(void)
int money;
printf("Enter the amount?");
scanf("%d", &money);
int notes[9] = {500, 200, 100, 50, 20, 10, 5, 2, 1};
printf("cashier gives\n");

for (int i=0; i<9; i++)
{
while (money >= notes[i])
{
printf("%d\n", notes[i]);
money -= notes[i];
}
}
}
```

Evolution of Programming language.

Programming languages are the tools that allow humans to communicate instructions to the computer.
Their evolution reflects the changing needs of technology, from simple calculations to complex artificial intelligence and web applications.

Machine level language:- It is the lowest level language consisting of binary codes (0 & 1).
It can only be understand by computers CPU, not by humans
It is the fastest programming language.

Assembly level language:- It is a platform/hardware dependent low level language that uses short codes to represent machine instructions.
It is readable than machine level language but requires a deep understanding of computer architecture.

High-level level:- It is a platform independent language.
This language are designed to be more human readable and abstract away many of the complexities of computer hardware Ex:- C, C++, Java, Python.

Q:- Why C is a structured programming language?
Ans:- (i) The entire C program in broken into small modules or functions
(ii) Each function perform a specific task, making the code organised.
(iii) C uses control statements.
(iv) Code reuseability
(v) Improved readability & maintainance.
(vi) Structured Data types.
(vii) C programs are built using three basic programming constructs sequence, selection and iteration.

Algorithm

* It is a well defined step by step set of instructions designed to solve a specific problem.

* It has well defined input/output.

* It has steps.

* It has no syntax. The focus is on correctness and efficiency of steps

* Used to define a clear, correct and finite method to solve a problem.

START → STOP

* Readable to all audiences

Pseudo Code

* Pseudo code is a textual representation of an algorithm.

* It is a textual, high-level and informal description of an algorithm's operating principle.

* It acts as a bridge b/w algorithm and actual code.

* has no syntax rules but follows the structural conventions of programming language (eg. if, else, while, for).

* Used to design and communicate the logic of a program.

BEGIN → END

* Readable to Programmers.

IDLE (Python), IDE, Thonny.

Ques: Write a code to print fibbonacci series?

Ques: Write a code to find simple Interest?

Ans (i)

```
# include <stdio.h>
void main(void)
{
int n, first=0, second=1, next, i;
printf("Enter the no. of toms?");
scanf("%d",&n);  printf("fibbonacci series");
for (i=0; i<n; i++)
 {
 if (i<=1)
 next=i;
 else
 {
 next= first +second;
 first= second;
   second= next;
   }
   printf("%d", next);
   }.
```

Ans (ii)

```
# include <stdio.h>
int main()
{
float principle, rate, time, SI;
printf("Enter the values?");
scanf("%f %f %f", &principle, &rate, &time);
SI= (principle * rate * time)/100;
printf(" Simple Interest= %.2f\n", SI);
   return 0;
   }
```

Ques:- Write a function to calculate square of a number entered by the user.

Ans:- M·I:- Function is accepting number and returning square

```
# include <stdio.h>
  int square (int); // function Prototype
   void main(void)
     {
     int a;
      printf ("Enter a number?");
      scanf ("%d", &a);
      printf ("The square of %d is %d\n", a, square(a));
// square (a) → actual arguments.
     }
     int square (int x) // formal arguments
      {
      return x* x;
      }.
```

M·II:- Function accepting a value and returning nothing.

```
  # include <stdio.h>
       void square (int); // function Prototype.
void main (void)
  {
  int a;
  printf ("Enter a number?");
     scanf ("%d", &a);
     Square (a)  // square (a) → actual arguments
  }
  void square (int x) // formal arguments.
    {
    print f ("The square of %d is %d \n", x, x*x);
    }
```

M·III:- Function accepting nothing but returning a value.

```
# include <stdio.h>
int square (void); // function Prototype
 int a; // a is global variable
 void main (void)
   {
   printf (" Enter a number?");
     scanf ("%d", &a);
 printf ("The square of %d is %d\n", a, square());
 }
 int square (void) // formal arguments
 {
 return a* a;
    }
```

M·IV:- Function accepting nothing and returning nothing.

```
# include <stdio.h>
 void square (void);
 int a, b;
    void main(void)
 {
 printf (" Enter a number?");
  scanf ("%d", &a);
   square();
 printf ("The square of %d is %d\n", a, b);
  }
  void square (void)
    {
    b= a* a;
     }
```

using a function swap two numbers?

Ans:- M.E:-

```
# include <stdio.h>
void swap(int, int); // function prototype
void main(void)
{
int a,b;
printf("Enter two numbers?");
scanf("%d %d",&a,&b);
swap(a,b); // function call by value
printf("\n In the main the value of a & b are in
                %d and %d \n", a,b);
}
void swap(int x, int y)
{
int temp;
temp=x;
x=y;
y=temp;
printf("\n In function swap values are %d and %d \n",x,y);
}
```

M.E:-

```
# include <stdio.h>
void swap(int*, int*)
void main(void)
{
int a,b;
printf("Enter two nos?");
scanf("%d %d",&a,&b);
swap(&a,&b);
printf("\n In main the value of a & b are %d and %d \n",
                                        a,b);
}
void swap(int *x, int *y)
{
int temp;
*x=*y;
*y=temp;
printf("\n In function swap the values are %d and %d \n",
                                        *x,*y);
}
```

Ques:- Ask the user to enter 10 integers using a function Calculate the sum and average of the numbers?

Ans:- M.E:- without function.

```
# include <stdio.h>
void main(void)
{
int a[10];
int i;
int sum=0;
float avg;
for(i=0;i<10;i++)
{
printf("Enter a[%d]?",i);
scanf("%d",&a[i]);
sum=sum+a[i];
}
avg=(float)sum/10;
printf("The sum is %d and avg is %f \n", sum, avg);
}
```

M.E:- with use of function.

```
# include <stdio.h>
void sum_avg(int*);
void main(void)
{
int a[10];
int i;
for(i=0;i<10;i++)
{
```

```
printf(" Enter a[%d]? ", i);
scanf(" %d", &a[i]);
}
sumavg(&a[0]);
}
void sumavg (int *p)
{
int j;
int sum=0;
float avg;
for (j=0; j<10; j++)
{
sum= sum + (*p);
p++;
}
avg= (float) sum/10;
printf(" The sum is %d and avg is %f \n", sum, avg);
}
```